Implementación de Modelos AI: Multiagente & Agente Simple

Despliegue en FastAPI con Docker y evaluación de enfoques.

M. en C. Gabriela Durán Meza.

Agenda

1. Objetivos
2. Contexto del proyecto entropía.ai
3. Implementación Multiagente
4. Implementación Agente Simple
5. Benchmark de Validación
6. Resultados del Benchmark
7. Casos de Uso y Recomendaciones
8. Próximos Pasos

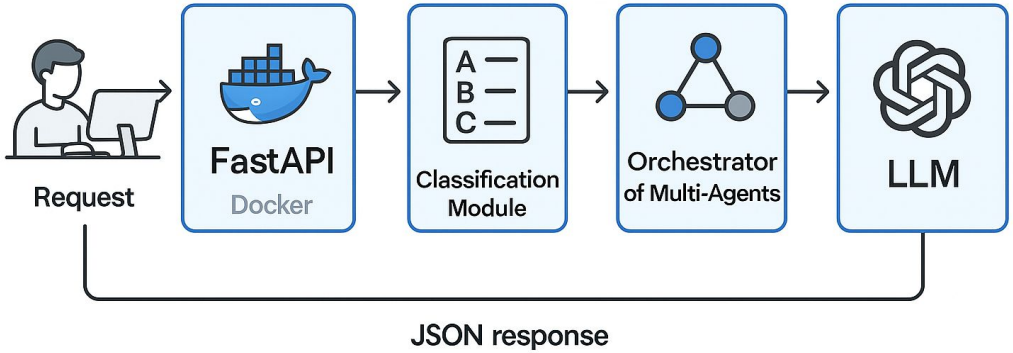
Objetivos del Proyecto

- Desarrollar e implementar un modelo Multiagente desplegado en FastAPI dockerizado.
- Multiagente: Orquestador de Agentes especializados por categoría (Legal, Contable, Médica, Genérica).
- Diseño de un Single-Agent con GPT-4o para casos de uso sencillos.
- Benchmark para decidir el enfoque de Agente Simple o Multiagente.

Contexto del proyecto entropía.ai

El flujo es:

1. Usuario envía la solicitud.
2. Llega a FastAPI ejecutándose dentro de Docker.
3. Pasa al Módulo de Clasificación de Categorías.
4. El Orquestador de Agentes distribuye la tarea al LLM apropiado.
5. El LLM (GPT-4o) genera la respuesta.
6. Se envía un JSON de vuelta al cliente.



El endpoint:

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/qa' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "question": "¿Cuál es el procedimiento contable para aplicar deducciones en la declaración anual?"
}'
```

Request URL

```
http://127.0.0.1:8000/qa
```

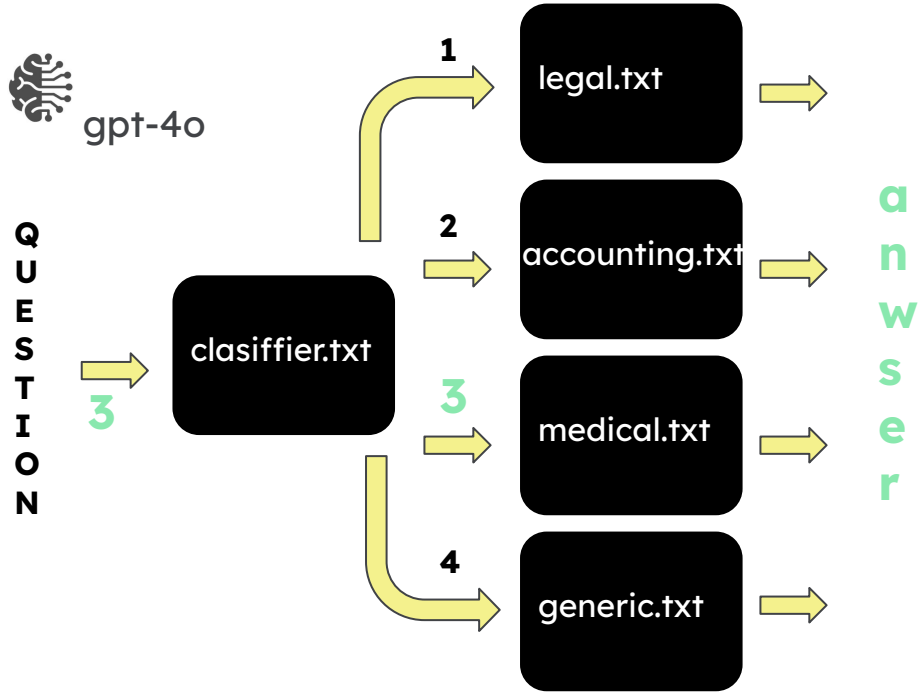
Server response

Code	Details
200	Response body


```
{
  "question": "¿Cuál es el procedimiento contable para aplicar deducciones en la declaración anual?",
  "category_id": 2,
  "category_label": "Pregunta Contable",
  "answer": "Para aplicar deducciones en la declaración anual, debes registrar en tu contabilidad los gastos estrictamente indispensables para la actividad, con comprobantes fiscales digitales (CFDI) válidos. Al elaborar la declaración, se incorporan estas deducciones autorizadas conforme al artículo 25 de la LISR, asegurando que no excedan los límites establecidos. Es fundamental conservar la documentación soporte y verificar que las deducciones estén correctamente clasificadas y registradas para su validación por el SAT."
}
```


Download

Multiagente



Estrategia secuencial-ramificada

classifier → nodo de dominio → END.

No se disparan varios LLM en paralelo; solo se ejecuta el nodo correspondiente a la categoría detectada.

- **Separación clara de responsabilidades**: Define estados y nodos con **langchain** y **langGraph**, el número de categoría decide qué nodo provee la respuesta.

- **Optimización de contextos complejos:** Se implementan cinco prompts especializados: uno por categoría.

- **Temperaturas diferenciadas**: *0.0* para clasificación (exactitud) y *0.2* para generación (creatividad moderada).

Agente Simple

Se proponen dos enfoques:

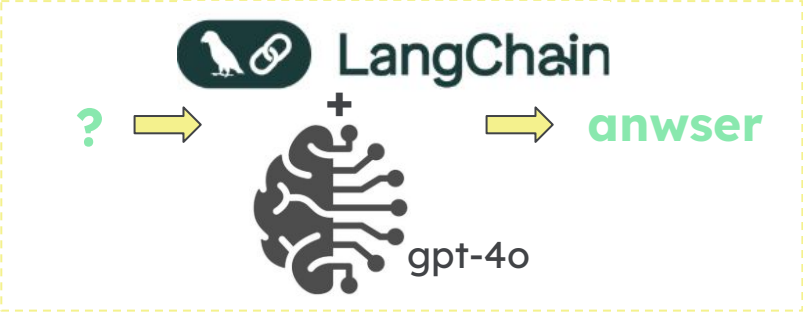
1. Agente Simple + prompt unificado

sólo 1 prompt concatenado

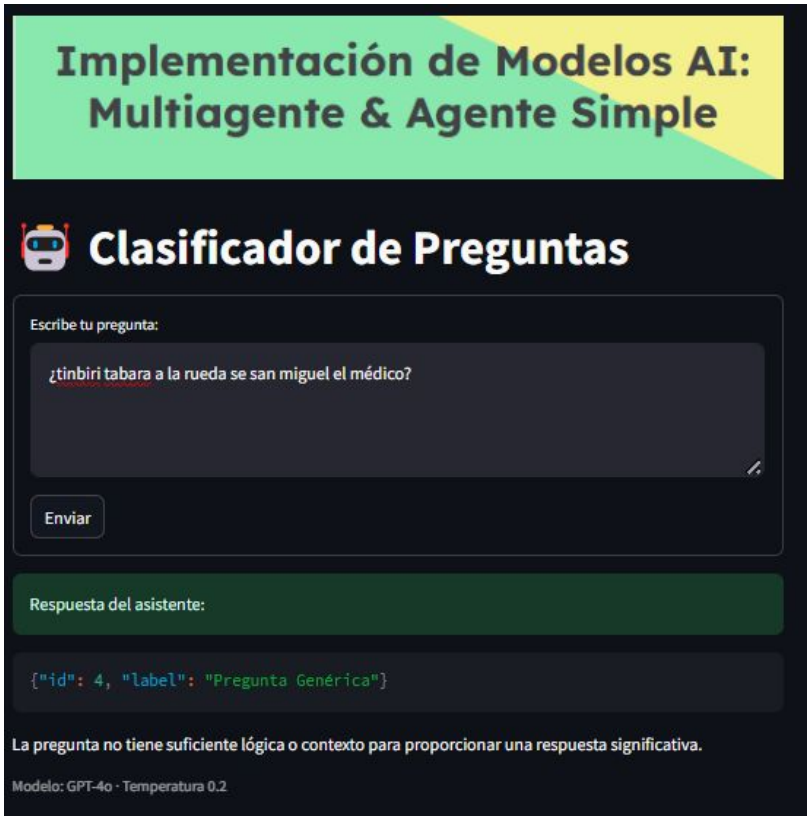
2. Agente Simple + prompt inteligente

Redacción simplificada del prompt

En una arquitectura simplificada:



Usando streamlit como interfaz de usuario:



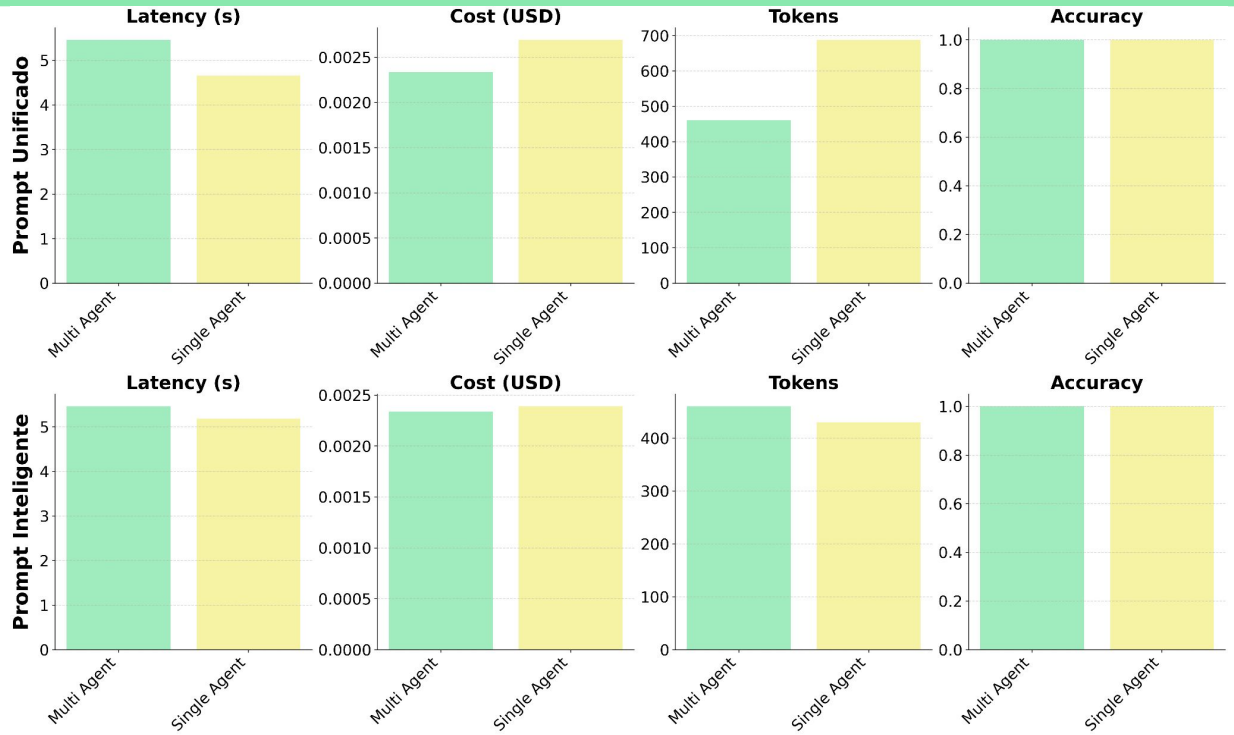
Benchmark de Validación

- Se consideran las métricas: latencia (s), costo(USD), número de tokens (input + output) y accuracy.

- En todos los enfoques se usa el modelo GPT-4o por las siguientes ventajas:
  - Manejo eficiente de contextos largos.
  - Alta precisión en clasificación y generación.
  - Balance entre latencia y costo.
  - Recurso disponible para la elaboración de este proyecto.

- Se comparan las métricas clave entre Multiagente vs Agente Simple (con ambos enfoques + prompt inteligente o +prompt unificado)

Resultados del Benchmark



- Mejor agilidad: El **agente simple** (Single Agent) ofrece **latencias significativamente menores** ~15 % en Prompt Unificado (PU), ~5 % en Prompt Inteligente (PI), mejora la agilidad en consultas puntuales.

- El agente simple tiene **costo por token ligeramente más alto** (+15 % en PU, +2 % en PI), puede subir (hasta +50 % en Prompt Unificado), regular los tokens de salida podría generar un **costo por token menor.**

- El **multiagente** logra **consumir un 6 % menos de tokens** frente al simple, reduciendo costes a gran escala y su arquitectura permite **escalar por dominio** (legal, contable, médico, genérico)

- Aunque su latencia es ligeramente superior, esta penalización se diluye cuando (a) el número de peticiones crece y (b) se paralelizan los sub-llamados internos.

Casos de uso y recomendaciones

Multiagente:

- **Análisis automático de contratos y documentos legales extensos:** Clasificador detecta si es "fusión/adquisición", "arrendamiento", etc. Cada tipo de documento tiene su propio prompt (financiero, inmobiliario, laboral).

- **Investigación médica y revisión de expedientes clínicos:** Clasificador detecta "diagnóstico", "medicación", "resultado de laboratorio". Agentes médicos generan resúmenes por sección y destacan riesgos (interacciones, alergias).

Agente

- **Chatbot de FAQ estáticas:** Responde preguntas frecuentes sobre un producto o servicio (horarios, políticas de devolución, contacto).
  ¿Por qué simple?: La base de conocimiento cabe en unos cuantos prompts o en un documento corto; no necesitas recuperación por vector DB ni chunking.

- Es útil para prototipos rápidos, primeros pilotos o entornos con pocos usuarios concurrentes, el agente simple es la opción óptima por simplicidad de despliegue y menor latencia.

Próximos pasos

- Ajustar el prompt del Agente Simple PI para generar salidas cortas y disminuir el costo por token.

- Implementar RAG en el multiagente para obtener mejor precisión en cada agente especializado (si se quiere alimentar un contexto muy grande).

- Implementar un sistema de monitorización en producción: Medir y alertar sobre métricas clave (latencia, coste por solicitud, precisión).

- Configurar CI/CD para asegurar calidad y despliegues automáticos: LangSmith (Local), Azure, AWS o GCP para orquestar el pipeline en la nube.

Conclusión

- Para prototipos rápidos, primeros pilotos o entornos con pocos usuarios concurrentes, el **Agente Simple PI** es la opción óptima por simplicidad de despliegue y menor latencia.

- En sistemas de misión crítica con alto volumen de usuarios o necesidades de especialización por dominio, el **Multiagente** compensa la latencia extra con mejor eficiencia de tokens y mayor flexibilidad operativa.

- La arquitectura del **Multiagente** permite escalar por dominio (legal, contable, médico, genérico), facilitando mantenimientos, actualizaciones de cada prompt y despliegue incremental de nuevas “sub-agentes”.

- El **Multiagente** y el **Agente Simple** mantienen la precisión al 100 %.

Apéndice

Las **pruebas unitarias** se dejaron al final, se propuso un script pero no está finalizado. Sin embargo, se pretende realizar al menos los siguientes test:

- Configuración del entorno: añade la carpeta padre al PYTHONPATH para poder importar app.main_multi_agent.
- Test de éxito: envía una pregunta válida a POST /qa y comprueba que responde con código 200 y el JSON esperado (categoría y respuesta simulada).
- Test de pregunta vacía: envía una cadena vacía (solo espacios) y verifica que responde con código 400 y el detalle “Empty question”.
- Test de classify_node: comprueba que, dado un JSON con "id", la función extrae correctamente category_id y asigna la etiqueta por defecto.
- Test de router: verifica que cada category_id se mapea al nombre de nodo siguiente correcto (“legal”, “accounting”, “medical” o “generic”).
- Test de domain_node: asegura que el nodo de dominio inserta en el estado la clave "answer" con el valor que devuelve la cadena simulada.
-

¡Gracias por su atención!